

Enhancing Consistency Models for Multi-Agent Trajectory Prediction

Alen Mrdovic¹, Qingze (Tony) Liu¹, Danrui Li¹, Mathew Schwartz^{1,2}, Kaidong Hu¹, Sejong Yoon³, Mubbassir Kapadia¹, and Vladimir Pavlovic¹

¹ Rutgers University - New Brunswick

² New Jersey Institute of Technology

³ The College of New Jersey

Abstract. Diffusion models for multi-agent trajectory prediction are limited by iterative denoising, which causes inference latency that hinders their use in time-critical settings like autonomous driving. Fast-sampling variants using DDIM and informed initial noise distributions partially alleviate this issue, but they either fail to achieve true single-step generation or are constrained by the chosen noise distribution. Consistency Models (CMs) offer high-quality one-step generation by mapping noise directly to data, but are difficult to train from scratch. We propose *EC-Traj*, an enhanced CM pipeline with improved training and conditional generation for trajectory prediction. Our framework extends the student-teacher consistency training scheme: the student produces standard outputs, while the teacher explicitly fuses its predictions with parts of the ground truth to give stronger supervision. We also exploit CMs’ direct denoising for top-K multi-shot generation during training. Combining conditional generation with this enhanced consistency objective yields faster inference and improved prediction accuracy, establishing competitive new benchmarks on the large-scale Argoverse 2 dataset.

Keywords: Trajectory prediction · Consistency models

1 Introduction

Trajectory prediction forecasts the short-term future motion of surrounding traffic participants from their observed history, environmental layout, and social interactions. It is vital for safety-critical applications such as robotic motion planning [15, 21, 23] and autonomous driving [16, 18], where predicted motions are required to be physically plausible, socially acceptable, and reflect multiple potential outcomes of participants’ intents. Recent work using diffusion models for trajectory prediction [6, 8, 17, 31] has shown strong performance in the multi-modal setting and the flexibility to impose various sampling techniques to meet additional output constraints. However, high computational complexity hinders the application of diffusion models, where real-time inference is crucial during deployment in systems such as autonomous vehicles.

During inference, diffusion models gradually denoise a standard Gaussian sample toward the desired data point by following dynamics specified by a

stochastic differential equation (SDE), a process that typically demands hundreds of optimization steps. More efficient variants, such as DDIM [24], speed up sampling and cut down the number of required steps; however, they still need more than 10 denoising iterations. Beyond accelerated diffusion formulations, previous work [17, 31] also tried to replace the standard Gaussian with an informed initial distribution. These methods denoise from a lower-variance noise distribution whose means come from trajectory predictions of pre-trained or standalone modules. This warm start reduces the required diffusion steps and promotes a better exploration of multi-modal futures using additional prior knowledge [12]. However, such explicit usage makes the model heavily dependent on the pre-trained predictor, as denoising from an inaccurate prior hinders the model from learning the correct denoising path.

Consistency Models (CMs) [27] enable single-step sampling at inference without relying on prior-informed sampling. Typically, CMs are either distilled from pre-trained diffusion models or trained from scratch, where the first approach achieves strong results in image generation. However, distillation requires robust pre-trained diffusion models, which are found scarce in trajectory prediction domain (shown later in this paper). The latter train-from-scratch method, on the other hand, requires carefully-tailored training guidance to ensure the quality.

We propose **ECTraj**, a conditional CM-based trajectory prediction framework. In line with a train-from-scratch paradigm, ECTraj is conditioned on both historical observations and a pre-trained prior, while performing denoising from a standard Gaussian distribution—thereby maximizing multi-modal exploration without being restricted by an informed initial distribution—and incorporates an improved consistency training pipeline. Our main contributions can be summarized as follows:

1. Instead of direct distillation or prior-informed sampling, we guide the training process by first using pretrained model components to encode the input into latent space, and then enhancing the student-teacher training with a ground-truth fusion to the teacher’s output.
2. Tailoring for the multi-modal nature of the trajectory predictions, we exploit one-step denoising to sample multiple modes for student and teacher, compute a best-of-K loss for each network; this optimization is new for CMs and not directly applicable to noise-predicting diffusion models.
3. ECTraj outperforms the diffusion baseline OptTrajDiff on all standard trajectory benchmark metrics in the large-scale Argoverse 2 dataset, while using only 1/10 of the inference steps.

2 Related Work

2.1 Diffusion Models for Trajectory Prediction

Diffusion models [7, 28] have achieved impressive results in image generation. More recently, they have been shown to be a promising framework for trajectory prediction. MID [6] captures the uncertainty of agent’s future movements

by modeling the process as conditional diffusion, where the encoded historical context serves as a condition to generate future trajectories. LED [17] speeds up the sampling process by using DDIM formulation and introducing a more informed prior different from standard Gaussian noise. This in turn reduces the number of sampling steps needed to generate high-quality trajectories. TrajDiffuse [13] transforms the prediction task as planning-based trajectory inpainting and introduces map-based guidance module to ensure environmental compliance in model output. MotionDiffuser [8] applies diffusion models to the task of multi-agent trajectory prediction for autonomous driving, where they incorporate a permutation-invariant denoiser to enhance multi-agent prediction. Opt-TrajDiff [31] is a multi-agent prediction method that, similarly to LED, forms an informed prior as a via pretrained QCNet [35] and proposed estimated clean manifold guidance to boost the performance in controllable generation. However, all existing approaches still require more than 10 denoising steps, which limits the practicality of deploying the model in time-sensitive applications.

2.2 Consistency Models

To alleviate the computational overhead of iterative inference, various fast-sampling diffusion techniques have been developed [9, 20, 24, 27]. Prominent among these are Consistency Models (CMs) [27], which achieve single-step generation from noise via a student-teacher training paradigm. CMs are typically optimized using one of two strategies: Consistency Distillation (CD) or Consistency Training (CT). While CD initializes the model from a pretrained diffusion model, CT trains the model entirely from scratch. Although CD initially outperformed CT by a wide margin, recent advancements [26] have systematically refined the design choices for CT, effectively bridging this performance gap. However, due to the lack of standardized pre-trained diffusion baselines for CD training, CMs have yet to be successfully adapted for the trajectory prediction domain. Inspired by recent extensions to the CM framework [2, 4, 5, 10], we propose ECTraj, a novel conditional CM-based trajectory prediction pipeline. ECTraj achieves state-of-the-art performance, while requiring only a tenth of the computational time compared to its diffusion counterparts.

3 Method

The overall architecture of our proposed model and its training scheme are depicted in Figure 1. We describe each component in detail below.

3.1 Consistency Models Preliminaries

Consistency models (CMs) are built on the Probability Flow ODE (PF-ODE) in continuous-time diffusion models [29], which defines a smooth bijective trajectory that transforms the data distribution $p_0(x_0)$ into a tractable noise distribution

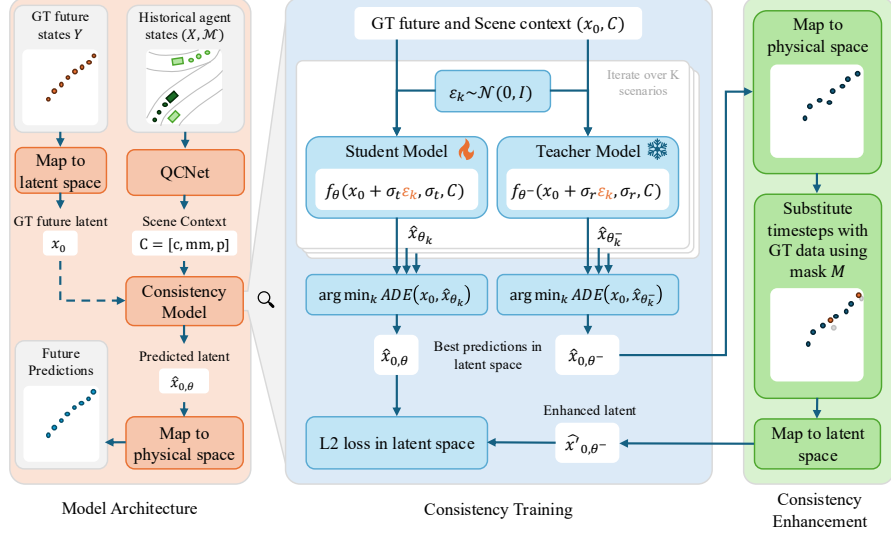


Fig. 1. Model Architecture and training scheme. (Left) The given historical trajectories and map information are encoded into a context latent vector. Then, the latent is processed by a consistency model and decoded to predicted future trajectories. (Middle) In the teacher-student training scheme of the consistency model, EC'Traj samples K different Gaussian noises to produce K different future trajectories. Then the student model is updated using the distance between the best student latent prediction and the best teacher latent prediction. (Right) To enable better teacher guidance, the predicted teacher latent is enhanced by replacing some time steps with GT data in physical space.

$p_{\sigma_T}(x_{\sigma_T}) \sim \mathcal{N}(0, \epsilon)$. CMs impose a self-consistency constraint so that points x_{σ_t} and x_{σ_r} on the same trajectory map to the same initial data point x_0 :

$$f(x_{\sigma_t}, \sigma_t) = f(x_{\sigma_r}, \sigma_r) = x_0,$$

enabling high-quality single-step generation from any intermediate state.

For training, the PF-ODE is discretized into time steps $t_1 < t_2 < \dots < t_N = T$. The associated variances $\sigma_1 < \sigma_2 < \dots < \sigma_N = T$ are obtained as a function of the time step, which is described in more detail in Section 3.3. CM then minimizes the distance between the denoised outputs of two adjacent steps:

$$\arg \min_{\theta} \mathbb{E}[w(\sigma_t) d(f_\theta(x_{\sigma_t}, \sigma_t), f_{\theta^-}(x_{\sigma_r}, \sigma_r))],$$

where $t_1 \leq r < t \leq T$. Here $d(\cdot, \cdot)$ is a distance metric, f_θ is the neural network estimating the consistency function, and f_{θ^-} is its exponential moving average (EMA).

3.2 Conditional CM for Multi-modal Trajectory Prediction

We propose a conditional CM framework for multi-agent trajectory prediction. A scene $(X, \mathcal{M})$ consists of agent states $X \in \mathbb{R}^{N_a \times T_h \times 2}$, where N_a is the number of agents, T_h the number of historical frames, and the last dimension the (x, y) coordinates, and a map $\mathcal{M} \in \mathbb{R}^{N_m \times D_p \times D_m}$, where N_m is the number of polylines, D_p the number of points per polyline, and D_m the number of attributes (e.g., position, orientation, magnitude). Given the encoded scene context $\mathbf{C}$, we learn a conditional consistency function $f_\theta(x_{\sigma_t}, \sigma_t, \mathbf{C})$ that directly maps a noisy prior to the future trajectory space to future predictions for $Y \in \mathbb{R}^{N_a \times T_f \times 2}$. To construct the conditional consistency function f_θ , we first encode the historical observations of the surrounding environment and social interactions into a context vector

$$c = \Phi(X, \mathcal{M}). \quad (1)$$

To account for the inherently multimodal nature of human motion, prior work typically introduces auxiliary modeling mechanisms that construct priors—either as deterministic anchor trajectories or as learned prior distributions—to facilitate exploration over multiple plausible futures. However, owing to the characteristics of consistency models (CM), which perform denoising directly from Gaussian white noise, it is nontrivial to decode trajectories directly from such priors. Consequently, we instead supply the consistency function with marginal future predictions and their associated scores, obtained from a pre-trained model, as prior anchor trajectory contexts, denoted by mm and p during training time to direct the model with more efficient learning.

$$mm, p = \Theta(c) \quad (2)$$

After forming the combined context representation $\mathbf{C} = [c, mm, p]$, we apply three layers of composite attention blocks to the intermediate noisy input x_σ . Within each layer, we first perform cross-attention between x_σ and the historical encodings of the target agent. We then apply cross-attention between x_σ and the map encodings (e.g., polygonal representations), followed by cross-attention with the context encodings of the neighboring agents. Subsequently, we conduct self-attention among the future encodings of different agents to capture their joint interactions. Finally, we introduce an additional cross-attention operation between the marginal priors and the noisy input, thereby integrating prior information into the denoising process and obtained the denoising output.

$$\hat{x}_0 = f_\theta(x_\sigma, \sigma, \mathbf{C}) \quad (3)$$

Input compression In the trajectory space, the flattened input has dimension $N_h \times 2$ (120 for Argoverse 2). Following [8] and [31], we accelerate the decoding by linearly compressing the trajectories into 10-dimensional latent vectors. The trajectory-level input X is mapped to latent input x via

$$x = XU, \quad (4)$$

and mapped back by

$$X = Vx. \quad (5)$$

The matrices U and V are learnable; details are given in the Appendix.

3.3 ECTraj - Enhanced Consistency training

Consistency formulation During training, we first sample the *student* index t from $\{1, \dots, N\}$, where N is the number of ODE trajectory discretization steps and determines the noise added to the clean input. Following [26], N is chosen adaptively. In ECTraj, we start with $N = 10$ and, over E training epochs, double N every $\frac{E}{3}$ epochs, so training ends with $N = 40$. The index t is sampled from a discrete log-normal distribution over indices, as in [26]; further details are in the Appendix. Given t , we then obtain the *teacher* index r as follows:

$$r = \lfloor t(1 - \frac{n(t')}{q^{\lceil \frac{e}{d} \rceil}}) \rfloor. \quad (6)$$

This parameterization is applied in line with ECT [5]. In Equation (6), $t' = \frac{t}{N}$, scaling t to $t' \in [0, 1]$, and $n(t') = 1 + \frac{k}{1+e^{bt'}}$; where $k = 8$ and $b = 1$. Additionally, e denotes the current epoch and d denotes a threshold epoch that is aligned with the discretization curriculum. More precisely, $d = E//3$ (where $//$ denotes integer division). In cases where $n(t') > q^{\lceil \frac{e}{d} \rceil}$, r is clamped to 0 to avoid negative indexing. The choice of q varies between domains and $q = 4$ was the best choice for our use case. This parameterization leads to a hybrid diffusion-consistency training procedure. In the beginning, the model essentially optimizes for reconstruction, since $r = 0$ in the initial stages of training due to clamping. As training progresses, r approaches t , transitioning the training process to standard consistency training [27] as training nears its end. Equation (6) describes the index retrieval. The noise to be added to the input is a function of the sampled index. For some index t , the variance is calculated in line with [9]:

$$\sigma_t = (\sigma_{min}^{\frac{1}{\rho}} + \frac{t-1}{N-1}(\sigma_{max}^{\frac{1}{\rho}} - \sigma_{min}^{\frac{1}{\rho}}))^{\rho}, \quad (7)$$

where $\sigma_{min} = 0.002$, $\sigma_{max} = 1.0^4$ and $\rho = 7$. For the purposes of discrete log-normal sampling, we also left-pad $\sigma_0 = 0.0$. The explanation for prepending this value is deferred to the Appendix. The student and teacher samples are obtained as:

$$x_{\sigma_t} = x_0 + \sigma_t \epsilon, \quad (8)$$

$$x_{\sigma_r} = x_0 + \sigma_r \epsilon, \quad (9)$$

where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, with $\mathbf{I}$ being the identity matrix. We point out that x_0 are compressed trajectory vectors and that optimization is done in the latent space.

⁴ This is different from the standard training for image generation where $\sigma_{max} = 80$. This value proved to be too large for our use case.

Multi-shot generation at training time. Our model produces high-quality trajectories in a single forward pass, which allows us to define a top- K consistency loss. This type of loss is not practical for diffusion-based methods such as OptTrajDiff [31], which operate in the noise space, making it non-trivial to identify the best trajectory. We demonstrate this empirically in Section 4.3. Specifically, we sample K random noise vectors and add them to both the student and the teacher:

$$x_{\sigma_t, k} = x_0 + \sigma_t \epsilon_k, \quad (10)$$

$$x_{\sigma_r, k} = x_0 + \sigma_r \epsilon_k, \quad (11)$$

for all $k \in \{1, 2, \dots, K\}$. $\epsilon_k \sim \mathcal{N}(0, I)$ represents K different random noise vectors. Next, we separately denoise the trajectories from each group (K student and K teacher modes). After denoising, we choose the best trajectories from both groups based on ADE :

$$k_\theta = \arg \min_k ADE(f_\theta(x_{\sigma_t, k}, \sigma_t, \mathbf{C}), x_0), \quad (12)$$

$$k_{\theta^-} = \arg \min_k ADE(f_{\theta^-}(x_{\sigma_r, k}, \sigma_r, \mathbf{C}), x_0). \quad (13)$$

θ^- is obtained using the exponential moving average (EMA) of θ . In line with [9], the denoiser f_θ is parameterized as follows:

$$f_\theta(x, \sigma, \mathbf{C}) = c_{skip}(\sigma)x + c_{out}(\sigma)F_\theta(x, \sigma, \mathbf{C}), \quad (14)$$

where F_θ denotes the trainable part of the denoiser (usually a neural network). The teacher parameterization is done similarly. $c_{skip}(\cdot)$ and $c_{out}(\cdot)$ denote differentiable functions for which $c_{skip}(\sigma_{min}) = 1$ and $c_{out}(\sigma_{min}) = 0$. The exact choices of these functions will be provided in the Appendix. For brevity, we will use the following notations:

$$\hat{x}_{0, \theta} = f_\theta(x_{\sigma_t, k_\theta}, \sigma_t, \mathbf{C}_t), \quad (15)$$

$$\hat{x}_{0, \theta^-} = f_{\theta^-}(x_{\sigma_r, k_{\theta^-}}, \sigma_r, \mathbf{C}_r). \quad (16)$$

Enhanced consistency objective Standard CM directly compares $\hat{x}_{0, \theta}$ and $\hat{x}_{0, \theta^-}$ from the student and teacher outputs. In ECTraj, we enhance the teacher output x_{σ_r} by fusing it with parts of the ground truth to provide stronger supervision for the student. Let X_f be the ground-truth future in trajectory space, and $\hat{X}_{0, \theta^-}$ the predicted trajectory-space output derived from $\hat{x}_{0, \theta^-}$ as described in Section 3.2. The modified teacher output is then defined as:

$$\hat{X}'_{0, \theta^-} = (1 - M) \odot \hat{X}_{0, \theta^-} + M \odot X_f. \quad (17)$$

Here, M denotes a binary mask that determines which parts of the ground truth are replacing the original output, and $\odot$ the Hadamard product. $\hat{X}_{0, \theta^-}$ is then

mapped back into the latent space representation $\hat{x}'_{0,\theta-}$, also as explained in Section 3.2.

We note the similarity between Equation (17) and the future mixup in TimeDiff [22], which can also be viewed as a variant of teacher forcing [32]. The key difference lies in the masking: TimeDiff uses random masking, while we deterministically replace two waypoints with ground truth—the midpoint and endpoint. Mixing these GT waypoints into the teacher output exposes the student network to higher quality supervision, as this simple modification refines the teacher output and provides the student with several ground truth anchors, thereby aiding learning. The ECTraj objective is:

$$\mathcal{L}_{ECTraj} = w(\sigma_t)d(\hat{x}_{0,\theta}, sg(\hat{x}'_{0,\theta-})), \quad (18)$$

where $w(\sigma_t) = \frac{1}{\sigma_t - \sigma_r}$, d is a distance metric (in our case, the L_2 norm), and sg is the stop-gradient operator applied to the teacher weights. Algorithm 1 shows the ECTraj training algorithm.

3.4 Inference

We design our inference procedure based on the recommended configuration from [26]. For multistep sampling, the intermediate time indices are defined as

$$\tau_n = \frac{n}{N}\tau_N, \quad (19)$$

where $n \in \{1, 2, \dots, N\}$. Here, τ_N is the largest index and corresponds to the maximum variance. ECTraj performs single-step denoising starting from standard Gaussian noise. In contrast to earlier diffusion-based methods [17, 31], ECTraj does not rely on an informed or specialized initial distribution. Although multistep inference is feasible, we empirically observed that single-step sampling yields the best performance (multi-step result in supplementary). We hypothesize that this is because multi-step inference excessively corrupts the samples through repeated noise injection, in line with the observations reported in [27]. Algorithm 2 shows the inference algorithm.

4 Experiments

Dataset and metrics. We conduct our experiments on the Argoverse 2 Motion Forecasting Dataset [33], a widely used large-scale benchmark for trajectory prediction. Each scenario provides 50 past time steps (5 seconds) and 60 future time steps (6 seconds). We adopt the standard metrics from the Argoverse 2 evaluation. Specifically, we use the accuracy metrics ADE_K and FDE_K with $K = 1$ and $K = 6$ to assess the upper bound of model prediction performance. We further report Brier- FDE_6 (abbreviated as $b-FDE_6$), which augments the

Algorithm 1 ECTraj Training

```

1: Input: Historical data  $(X_h, \mathcal{M})$ , ground-truth future data  $X_f$ , linear mapping  $U$ 
   and its inverse  $V$ , QCNet encoder/decoder  $(\Phi, \Theta)$ , max epochs  $E$ , discrete lognor-
   mal params  $(\mu, \Sigma)$ , modes  $K$ , EMA parameter  $\alpha$ , binary mask  $M$ 
2: Init: Randomly initialize  $\theta$  and  $\theta^-$ 
3: for  $e = 1, \dots, E$  do
4:   for  $((x_h, m), x_f)$  in  $((X_h, \mathcal{M}), X_f)$  do
5:      $\mathbf{c} = \Phi(x_h, m)$ 
6:      $(\mathbf{mm}, \mathbf{p}) = \Theta(\mathbf{c})$ 
7:      $N = 10 \cdot 2^{\lceil \frac{e}{3E} \rceil}$ 
8:      $t \sim \text{DiscreteLogNormal}(\mu, \Sigma, N)$ 
9:      $r = \lfloor t(1 - \frac{n(t')}{q^{\lceil \frac{t'}{2} \rceil}}) \rfloor$ 
10:     $x_0 = x_f U$ 
11:    for  $k = 1, \dots, K$  do
12:       $\varepsilon_k \sim \mathcal{N}(0, I)$ 
13:       $x_{\sigma_t, k} = x_0 + \sigma_t \varepsilon_k$ 
14:       $x_{\sigma_r, k} = x_0 + \sigma_r \varepsilon_k$ 
15:    end for
16:     $\mathbf{C} = (\mathbf{c}, \mathbf{mm}, \mathbf{p})$ 
17:     $k_\theta = \arg \min_k \text{ADE}(f_\theta(x_{\sigma_t, k}, \sigma_t, \mathbf{C}), x_0)$ 
18:     $k_\theta^- = \arg \min_k \text{ADE}(f_\theta^-(x_{\sigma_r, k}, \sigma_r, \mathbf{C}), x_0)$ 
19:     $\hat{x}_{0, \theta} = f_\theta(x_{\sigma_t, k_\theta}, \sigma_t, \mathbf{C})$ 
20:     $\hat{x}_{0, \theta^-} = f_\theta^-(x_{\sigma_r, k_{\theta^-}}, \sigma_r, \mathbf{C})$ 
21:     $\hat{X}_{0, \theta^-} = V x_{0, \theta^-}$ 
22:     $\hat{X}'_{0, \theta^-} = (1 - M) \odot \hat{X}_{0, \theta^-} + M \odot X_f$ 
23:     $\hat{x}'_{0, \theta^-} = \hat{X}'_{0, \theta^-} U$ 
24:     $\mathcal{L}_{ECTraj} = w(\sigma_t) d(\hat{x}_{0, \theta}, \hat{x}'_{0, \theta^-})$ 
25:     $\theta^- = \text{sg}((1 - \alpha) \theta + \alpha \theta^-)$   $\triangleright$  EMA update for teacher; sg = stop-gradient
26:  end for
27: end for
28: return  $\mathcal{L}_{ms}$ 

```

FDE with a confidence penalty term of $(1 - p^2)$, where p is the predicted probability of the best-of- K trajectory. In addition, we measure the miss rate (MR_K), defined as the fraction of scenarios in which none of the K predicted trajectories has an FDE within **2.0m**, and the collision rate (CR_K), defined as the fraction of scenarios in which at least two agents collide, using a collision distance threshold of **1.0m**.

Baselines. We compared our method with several recent state-of-the-art baselines whose results on Argoverse 2 are publicly available. Specifically we compare ECTraj to OptTrajDiff [31] as closest counterpart, which is the only diffusion-based model directly evaluating on Argoverse 2. We list the benchmark results in Table 1.

Algorithm 2 Inference

Input: Historical data $(X_h, \mathcal{M})$, noisy prior x_N , QCNet encoder/decoder Φ/Θ , learned consistency model θ , number of inference steps N , intermediate sampling timestep indices $\tau_0 < \tau_1 < \tau_2 < \dots < \tau_{N-1}$, such that $\sigma_{\tau_0} = 0$, $\sigma_{\tau_1} = \sigma_{min}$, $\sigma_{\tau_{N-1}} = \sigma_{max}$:

```

for  $(x_h, m)$  in  $(X_h, M)$  do
   $x_{next} = x_N$ 
   $\mathbf{c} = \Phi(x_h, m)$ 
   $(\mathbf{mm}, \mathbf{p}) = \Theta(\mathbf{c})$ 
   $\epsilon \sim \mathcal{N}(0, I)$ 
   $\mathbf{C} = (\mathbf{c}, \mathbf{mm}, \mathbf{p})$ 
  for  $i$  in  $N-1..1$  do
     $x_{0, \tau_i} = f_{\theta}(x_{next}, \sigma_{\tau_i}, \mathbf{C})$ 
     $x_{next} = x_{0, \tau_i} + \sigma_{\tau_{i-1}} \epsilon$ 
  end for
end for
return  $x_{next}$ 

```

Training details. Our model was trained for 60 epochs, using four RTX A6000 GPUs. During training, we used the batch size of 16, and AdamW [14] optimizer with the initial learning rate of 0.002 and the weight decay of 0.0001. For multi-shot trajectory generation, we generated $K = 6$ future joint trajectories for each scene in the training set, since metric evaluation is commonly done using this number of modes.

4.1 Quantitative Result: Argoverse 2

Our main experiment consists of comparing ECTraj with several state-of-the-art baselines. The results are shown in Table 1. ECTraj achieves state-of-the-art result for FDE_6 , and is on par with QCNeXt for all metrics. ECTraj achieves better performance than OptTrajDiff on all metrics, showcasing the effectiveness of our CM formulation. Although ECTraj is only the third in terms of FDE_1 , the runner-up method in terms of this metric (FutureNet-LOF) achieves noticeably weaker accuracy metrics and miss rate for $K = 6$. We also demonstrate additional examples of ECTraj exhibiting better diversity in Section 4.2.

Inference speed One major motivation for replacing the diffusion denoiser with a consistency one lies in its inference speed. We compare our method with OptTrajDiff [31] in this aspect and show that our method achieves up to 10 times faster inference compared to OptTrajDiff in Table 2. OptTrajDiff uses 10 inference steps, while ECTraj uses only one, which we report under the NFE (Number of **F**unction **E**valuations) column. As a more objective measure of inference speed compared to raw GPU time, we also report Giga-floating point operations (GFLOPs). Combined with the superior prediction performance, ECTraj demonstrates the strong potential using CM as alternative backbone for trajectory prediction task.

Table 1. Standard Argoverse2 metric evaluation

Method	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
FJMP [19]	0.81	1.89	1.52	4.00	2.59	0.19	0.01
GNet [3]	0.69	1.46	1.23	3.05	2.12	0.19	0.01
QCNeXt [36]	0.50	1.02	0.94	2.29	1.65	0.13	0.01
Forecast-MAE [1]	0.69	1.55	1.30	3.33	2.24	0.19	0.01
OptTrajDiff [31]	0.60	1.31	1.08	2.71	1.95	0.17	0.01
DeMo [34]	0.58	1.24	1.12	2.78	1.93	0.16	0.01
RealMotion [25]	0.62	1.32	1.14	2.87	2.01	0.18	0.01
FutureNet-LOF [30]	0.58	1.25	0.96	<u>2.34</u>	<u>1.68</u>	0.18	0.02
DONUT [11]	0.55	1.13	1.07	2.62	1.79	0.15	0.01
ECTraj (ours)	0.50	0.96	0.94	2.37	<u>1.68</u>	0.13	0.01

Table 2. Inference speed of ECTraj

Method	NFE	GFLOPs
OptTrajDiff	10	~ 1311
ECTraj	1	~ 132

4.2 Qualitative examples

In Figure 2, we present visual results across the following four methods: *QCNeXt* [36], *OptTrajDiff* [31], *ECTraj* (*one-shot loss*) and *ECTraj* (*six-shot loss*). In these scenarios, ECTraj excels at various aspects of trajectory prediction:

- **Scenario 1 - diversity and environmental compliance**; although all three models are able to model diverse movements, only ECTraj is able to predict an alternate mode that goes straight ahead without violating environmental constraints. Additionally, six-shot ECTraj is able to generate trajectories that better align with the ground truth compared to the one-shot variant.
- **Scenario 2 - accuracy**; although all three methods overshoot the ground truth goal point, ECTraj does so to the smallest extent. In this scenario, accuracy-wise, there is no significant difference between one-shot and six-shot ECTraj. This may be due to the simple setting of this scenario, which does not allow for many socially compliant types of intent (i.e. the only feasible intents are going straight ahead or stopping)
- **Scenario 3 - complex movements**; although all three methods recognize the intent of a sharp left turn, both QCNeXt and OptTrajDiff predict some modes which pass a non-traversable area, which does not happen in ECTraj. This is even more apparent in the six-shot setting.

In summary, the qualitative comparisons clearly establish the proposed ECTraj along with the multi-shot loss as the superior approach. While training with one-shot loss provides a competent baseline for simpler scenarios, it is the six-shot training formulation that unlocks the full robust capability of our method.

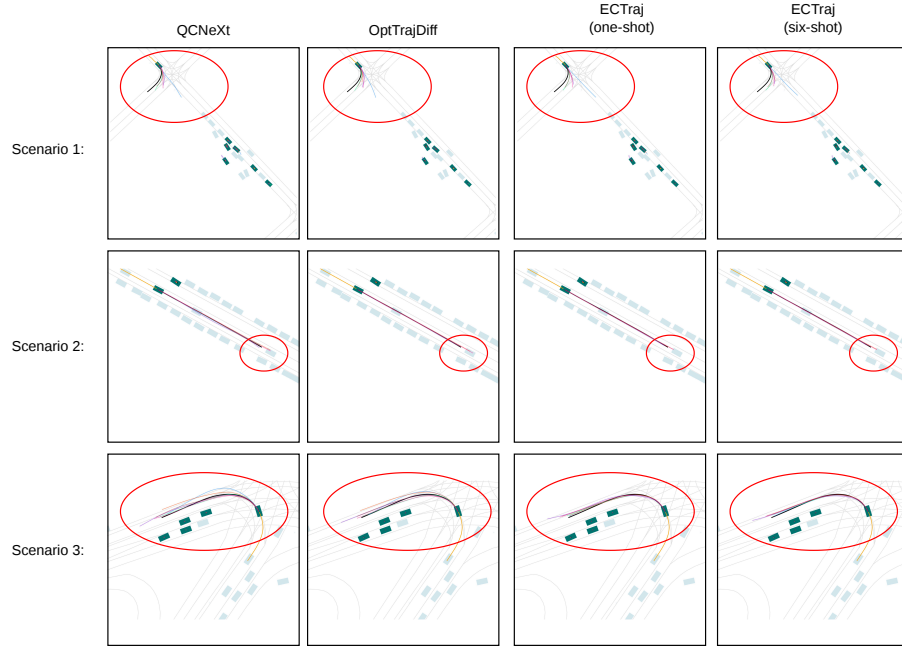


Fig. 2. Qualitative examples. Historical trajectories are denoted in **orange**, and ground truth future is denoted in **black**. Each of the 6 predicted modes is coded in different colors for easier interpretability. Regions of interest in each scenarios are encircled in **red**, also for easier interpretability.

By improving alignment with ground-truth trajectories and better adherence to environmental constraints during complex maneuvers, ECTraj demonstrate enhanced generative quality with reduced computation latency.

4.3 Ablation study

In this section, we analyze the impact of enhanced consistency training designs, top-K training in Diffusion formulation, and CM scheduling choices.

Enhanced consistency objective With results shown in Table 3, all our training components contribute to the model’s performance. As indicated in row 1 of Table 3, one-shot training shows reduced performance, which aligns with the observation from qualitative analysis. We also ablate the teacher output enhancement in two ways:

- **Standard CM objective** with no fusion of ground truth with teacher outputs.
- **Reconstruction objective:** We replace every single teacher output point with the ground truth, which reduces the method to the standard direct denoising objective in some variants of diffusion models [9].

Table 3. Ablation study on the individual components of ECTraj. The ablated components denote the following: (1) *one-shot training* - replacing six-shot mode generation at training with one-shot, (2) *standard CM objective* - removing fusion of ground truth with teacher, (3) *reconstruction objective* - changing teacher output with ground truth (full ground truth fusion).

Ablated component	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
One-shot training	0.53	1.03	0.98	2.39	1.76	0.14	0.007
Standard CM objective	0.52	0.99	0.96	2.38	1.70	0.14	0.007
Reconstruction objective	0.57	1.10	1.08	2.53	1.87	0.17	0.009
ECTraj	0.50	0.96	0.94	2.37	1.68	0.13	0.006

Both ablations yield suboptimal performance on all metrics. This is intuitive for the following reasons: with the *standard CM objective*, although this objective is the easiest for the student model to learn, it can be unreliable because the teacher model itself may produce inaccurate outputs. With the *reconstruction objective*, the teacher’s output is, in principle, as accurate as possible, but it becomes difficult for the student to learn from—especially when trained with a small number of steps, as in ECTraj. In this case, the student model would require many more iterations to converge. Consequently, the ECTraj training pipeline strikes a careful balance between the quality of supervision and the ease of optimization.

One-shot vs. multi-shot generation at training time As demonstrated in the ablation study above, introducing multiple diverse modes facilitates faster learning, as the model can learn dedicated denoising trajectories for distinct types of motion. In diffusion-based methods such as OptTrajDiff, the best-of-K strategy is not directly applicable: because diffusion models are trained to predict noise, it is not evident how to define or select the “best” noisy mode. Consequently, we investigate two alternative formulations to analyze the impact of multi-shot loss training on the denoising process.

- **Variant 1:** the best noisy mode is selected as the mode that is the closest to the ground truth **data**
- **Variant 2:** the best noisy mode is selected as the mode that is the closest to the ground truth **noise**

We studied the performance of these two variants in Table 4. Although **Variant 1** does outperform OptTrajDiff in terms of metrics FDE_6 and is on par with it in terms of MR_6 , both variants result in substantially worse metric values compared to ECTraj.

Consistency scheduling type In ECTraj, we use timestep scheduling in line with ECT [5]. In image generation, this method was shown to surpass previous baselines such as [27] and [26]. This is also the case in trajectory prediction,

Table 4. Best-of-K approach used in ECTraj tested on OptTrajDiff

Variant	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
Variant 1	0.64	1.17	1.44	3.46	2.02	0.17	0.01
Variant 2	0.99	1.42	2.38	5.10	2.30	0.21	0.03
OptTrajDiff (one-shot)	0.60	1.31	1.08	2.71	1.95	0.17	0.01
ECTraj	0.50	0.96	0.94	2.37	1.68	0.13	0.01

Table 5. ICT vs ECT scheduling

Scheduling Type	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
ICT	0.52	1.02	0.95	2.38	1.77	0.15	0.006
ECT (ECTraj)	0.50	0.96	0.94	2.37	1.68	0.13	0.006

as we found our ECT scheduling gains a small improvement compared to prior practice (ICT scheduling) [26] in Table 5.

5 Conclusion

In this paper, we presented **ECTraj**, a multi-agent trajectory prediction model which utilizes consistency models. We depart from the standard notion of consistency models, and allow fuse parts of the teacher output with the ground truth, where we use the midpoint and the endpoint of the ground truth future to promote accuracy. On top of that, we incorporate a best-of-K training process, which is not feasible in diffusion-based baseline which optimize for noise prediction (such as OptTrajDiff). Both of these components help ECTraj achieve state-of-the-art results on almost every Argoverse 2 metric, which is particularly noticeable with FDE_6 .

5.1 Limitations and future work

One limitation of ECTraj along with previous diffusion counterparts is its reliance on marginal QCNet priors. In the future work, we will consider an end-to-end approach that either doesn't depend on these priors or uses self-obtained priors.

References

1. Cheng, J., Mei, X., Liu, M.: Forecast-mae: Self-supervised pre-training for motion forecasting with masked autoencoders. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). pp. 8679–8689 (2023)
2. Frans, K., Hafner, D., Levine, S., Abbeel, P.: One step diffusion via shortcut models. In: The Thirteenth International Conference on Learning Representations (ICLR) (2025)
3. Gao, X., Jia, X., Li, Y., Xiong, H.: Dynamic scenario representation learning for motion forecasting with heterogeneous graph convolutional recurrent networks. IEEE Robotics and Automation Letters **8**(5), 2946–2953 (2023)
4. Geng, Z., Deng, M., Bai, X., Kolter, J.Z., He, K.: Mean flows for one-step generative modeling. In: The Thirty-ninth Annual Conference on Neural Information Processing Systems (NeurIPS) (2025)
5. Geng, Z., Pokle, A., Luo, W., Lin, J., Kolter, J.Z.: Consistency models made easy. In: The Thirteenth International Conference on Learning Representations (ICLR) (2025)
6. Gu, T., Chen, G., Li, J., Lin, C., Rao, Y., Zhou, J., Lu, J.: Stochastic trajectory prediction via motion indeterminacy diffusion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 17113–17122 (2022)
7. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. Advances in Neural Information Processing Systems (NeurIPS) **33**, 6840–6851 (2020)
8. Jiang, C., Cornman, A., Park, C., Sapp, B., Zhou, Y., Anguelov, D., et al.: Motion-Diffuser: Controllable multi-agent motion prediction using diffusion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 9644–9653 (2023)
9. Karras, T., Aittala, M., Aila, T., Laine, S.: Elucidating the design space of diffusion-based generative models. Advances in Neural Information Processing Systems (NeurIPS) **35**, 26565–26577 (2022)
10. Kim, D., Lai, C.H., Liao, W.H., Murata, N., Takida, Y., Uesaka, T., He, Y., Mitsufoji, Y., Ermon, S.: Consistency trajectory models: Learning probability flow ode trajectory of diffusion. In: The Twelfth International Conference on Learning Representations (ICLR) (2024)
11. Knoche, M., de Geus, D., Leibe, B.: Donut: A decoder-only model for trajectory prediction. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). pp. 28903–28912 (2025)
12. Li, C., Zhang, R., Huang, S., Zhong, X., Jiang, H.: Agma: Adaptive gaussian mixture anchors for prior-guided multimodal human trajectory forecasting. arXiv preprint arXiv:2602.04204 (2026)
13. Liu, Q.T., Li, D., Sohn, S.S., Yoon, S., Kapadia, M., Pavlovic, V.: TrajDiffuse: A conditional diffusion model for environment-aware trajectory prediction. In: International Conference on Pattern Recognition (ICPR). pp. 382–397. Springer (2024)
14. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2018)
15. Luo, J., Zhang, C., Si, W., Jiang, Y., Yang, C., Zeng, C.: A physical human–robot interaction framework for trajectory adaptation based on human motion prediction and adaptive impedance control. IEEE Transactions on Automation Science and Engineering **22**, 5072–5083 (2024)

16. Madjid, N.A., Ahmad, A., Mebrahtu, M., Babaa, Y., Nasser, A., Malik, S., Hassan, B., Werghi, N., Dias, J., Khonji, M.: Trajectory prediction for autonomous driving: Progress, limitations, and future directions. *Information Fusion* p. 103588 (2025)
17. Mao, W., Xu, C., Zhu, Q., Chen, S., Wang, Y.: Leapfrog diffusion model for stochastic trajectory prediction. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 5517–5526 (2023)
18. Raina, S., Challagundla, J., Singh, M.: Trajectory prediction in autonomous driving: A comprehensive review of deep learning models and future direction. In: *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*. pp. 00753–00759. IEEE (2025)
19. Rowe, L., Ethier, M., Dykhne, E.H., Czarnecki, K.: FJMP: Factorized joint multi-agent motion prediction over learned directed acyclic interaction graphs. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 13745–13755 (2023)
20. Salimans, T., Ho, J.: Progressive distillation for fast sampling of diffusion models. In: *International Conference on Learning Representations (ICLR)* (2022)
21. Samavi, S., Lem, A., Sato, F., Chen, S., Gu, Q., Yano, K., Schoellig, A.P., Shkurti, F.: Sicnav-diffusion: Safe and interactive crowd navigation with diffusion trajectory predictions. *IEEE Robotics and Automation Letters* (2025)
22. Shen, L., Kwok, J.: Non-autoregressive conditional diffusion models for time series prediction. In: *International Conference on Machine Learning (ICML)*. pp. 31016–31029. PMLR (2023)
23. Singha, A., Nanwani, L., Jain, S., Singamaneni, P.T., Singh, A.K., Krishna, K.M., et al.: Crowd-fm: Learned optimal selection of conditional flow matching-generated trajectories for crowd navigation. *arXiv preprint arXiv:2602.06698* (2026)
24. Song, J., Meng, C., Ermon, S.: Denoising diffusion implicit models. In: *International Conference on Learning Representations (ICLR)* (2021)
25. Song, N., Zhang, B., Zhu, X., Zhang, L.: Motion forecasting in continuous driving. *Advances in Neural Information Processing Systems (NeurIPS)* **37**, 78147–78168 (2024)
26. Song, Y., Dhariwal, P.: Improved techniques for training consistency models. In: *The Twelfth International Conference on Learning Representations (ICLR)* (2024)
27. Song, Y., Dhariwal, P., Chen, M., Sutskever, I.: Consistency models. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. pp. 32211–32252 (2023)
28. Song, Y., Ermon, S.: Generative modeling by estimating gradients of the data distribution. *Advances in Neural Information Processing Systems (NeurIPS)* **32** (2019)
29. Song, Y., Sohl-Dickstein, J., Kingma, D.P., Kumar, A., Ermon, S., Poole, B.: Score-based generative modeling through stochastic differential equations. In: *International Conference on Learning Representations (ICLR)* (2021)
30. Wang, M., Ren, X., Jin, R., Li, M., Zhang, X., Yu, C., Wang, M., Yang, W.: FutureNet-LOF: Joint trajectory prediction and lane occupancy field prediction with future context encoding. In: *2025 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 8841–8848. IEEE (2025)
31. Wang, Y., Tang, C., Sun, L., Rossi, S., Xie, Y., Peng, C., Hannagan, T., Sabatini, S., Poerio, N., Tomizuka, M., et al.: Optimizing diffusion models for joint trajectory prediction and controllable generation. In: *European Conference on Computer Vision (ECCV)*. pp. 324–341. Springer (2024)
32. Williams, R.J., Zipser, D.: A learning algorithm for continually running fully recurrent neural networks. *Neural Computation* **1**(2), 270–280 (1989)

33. Wilson, B., Qi, W., Agarwal, T., Lambert, J., Singh, J., Khandelwal, S., Pan, B., Kumar, R., Hartnett, A., Pontes, J.K., et al.: Argoverse 2: Next generation datasets for self-driving perception and forecasting. In: Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2) (2021)
34. Zhang, B., Song, N., Zhang, L.: Decoupling motion forecasting into directional intentions and dynamic states. *Advances in Neural Information Processing Systems (NeurIPS)* **37**, 106582–106606 (2024)
35. Zhou, Z., Wang, J., Li, Y.H., Huang, Y.K.: Query-centric trajectory prediction. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 17863–17873 (2023)
36. Zhou, Z., Wen, Z., Wang, J., Li, Y.H., Huang, Y.K.: QCNeXt: A next-generation framework for joint multi-agent trajectory prediction. *arXiv preprint arXiv:2306.10508* (2023)

ECTraj: Enhanced Consistency Training for Multi-Agent Trajectory Prediction - Appendix

Alen Mrdovic¹, Qingze (Tony) Liu¹, Danrui Li¹, Mathew Schwartz^{1,2}, Kaidong Hu¹, Sejong Yoon³, Mubbasir Kapadia¹, and Vladimir Pavlovic¹

¹ Rutgers University - New Brunswick

² New Jersey Institute of Technology

³ The College of New Jersey

1 Additional ablation studies

In this section, we will describe some additional experiments that could not fit in the main submission, which are useful for a more complete assessment of ECTraj.

1.1 Consistency training hyperparameters

In our work, we use the time step scheduling in line with [1]. The scheduling itself depends on several hyperparameters. We sample the student index t from the discrete lognormal distribution, which is explained in more detail in Section 2.2 of this Appendix. Then, we sample the teacher index r as follows:

$$r = \lfloor t(1 - \frac{n(t')}{q^{\lfloor \frac{t}{d} \rfloor}}) \rfloor. \quad (1)$$

We define

$$n(t') = \frac{k}{1 + e^{bt'}}, \quad t' = \frac{t}{N}, \quad t' \in [0, 1].$$

Following [1], we fix $k = 8$ and $b = 1$ for all experiments, making $n(t')$ independent of the dataset.

We set $d = \lfloor E/3 \rfloor$, where E is the maximum number of training epochs. This choice is aligned with our discretization curriculum: the number of discretization steps N doubles every $\lfloor E/3 \rfloor$ epochs. To keep the schedule consistent, we increase d in step with N .

The parameter q , in contrast, is dataset-dependent according to [1]. Since they evaluate on high-dimensional image data, whereas we work with 10-dimensional latent trajectory vectors, we must tune q for our setting. We therefore test $q \in \{2, 4, 6, 8\}$.

Table 1 reports the range of the teacher–student index ratio

$$\alpha = \frac{r}{t},$$

for different q and numbers of discretization steps N (shown as columns). When $\alpha < 0$, we clamp it to 0 to avoid negative teacher time-step indices.

Table 1: Range of teacher-to-student time step ratio $\alpha = \frac{\tau}{t}$, for each q and each number of discretization steps N

q	$N = 10$	$N = 20$	$N = 40$
2	$\alpha = 0$	$\alpha \in [0, 0.21]$	$\alpha \in [0.38, 0.60]$
4	$\alpha \in [0, 0.21]$	$\alpha \in [0.69, 0.80]$	$\alpha \in [0.92, 0.96]$
6	$\alpha \in [0.17, 0.48]$	$\alpha \in [0.86, 0.91]$	$\alpha \in [0.98, 0.99]$
8	$\alpha \in [0.38, 0.60]$	$\alpha \in [0.92, 0.96]$	$\alpha \in [0.98, 0.99]$

Table 2: Ablation on the q time step scheduling hyperparameter

q	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
2	0.51	0.96	1.01	2.44	1.68	0.14	0.007
4	0.50	0.96	0.94	2.37	1.68	0.13	0.006
6	0.52	1.04	0.99	2.50	1.77	0.14	0.007
8	0.54	1.09	1.01	2.47	1.82	0.15	0.007

Table 2 presents an ablation study over the four q values. While $q = 2$ performs reasonably well (especially on FDE_6 and $b-FDE_6$), $q = 4$ consistently yields the best overall results. For larger q , performance degrades across metrics. Consequently, we adopt $q = 4$ in all main experiments.

1.2 Ground truth fusion

In the main paper, we tested our final choice of ground truth fusion (midpoint + endpoint) against two other methods. Here, we will provide a more detailed overview of two more variants of fusion we tested our model with. We will provide an overview of each type of fusion here, followed by a corresponding abbreviation used in Table 3 for readability:

1. **No fusion (NF)**: Fusion is not applied to the teacher output.
2. **Full fusion (FF)**: The teacher output is fully replaced with ground truth - effectively yielding a direct denoising reconstruction objective similar to [3].
3. **Random 2 (R2)**: Random two ground truth points are selected to be fused with the teacher output. This is done in the spirit of TimeDiff [4], where a random binary mask is also used.
4. **Progressive (Prog)**: We start by fusing more evenly spaced points at the beginning of training, after which we gradually decrease the number of points (which are still evenly spaced) to fuse as the training progresses. In our case, we start by replacing **10** points, and decrease the number of points by **2** for every 10 epochs.
5. **Midpoint + Endpoint (M+E) (ECTraj)**: We only fuse the midpoint and the endpoint over the entire duration of training.

Table 3 gives an overview of each of these methods. Although some methods are very close to our final version of ECTraj, we see that the **M+E** fusion still

Table 3: Ablation on different types of ground truth fusion

Method	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
NF	0.52	0.99	0.96	2.38	1.70	0.14	0.007
FF	0.57	1.10	1.08	2.53	1.87	0.17	0.007
R2	0.50	1.00	0.98	2.47	1.70	0.14	0.007
Prog	0.51	1.05	0.96	2.39	1.73	0.15	0.006
M+E (ECTraj)	0.50	0.96	0.94	2.37	1.68	0.13	0.006

Table 4: Ablation on the number of sampling steps. NFE denotes the number of sampling steps

	NFE	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
1	0.50	0.96	0.94	2.37	1.68	0.13	0.006	
2	0.52	1.01	0.96	2.41	1.74	0.13	0.007	
4	0.54	1.03	0.96	2.42	1.77	0.14	0.007	

gives the best results across all metrics. This may be due to this method’s best interpolation between the two extremes:

1. **Full fusion:** Although in this case the “output” the loss is measured against is the ground truth, which is the most accurate output, learning this objective can be hard, since it requires huge traversals through the PF ODE [6], which can be difficult at the initial stages of training.
2. **No fusion:** Although this objective is the easiest to learn due to the proximity of the outputs, it may be inaccurate, since it depends on the teacher model which in itself may be inaccurate, especially at the early stages of training [5, 6].

1.3 Multi-step sampling

In the main paper, we use single-step sampling for all reported ECTraj results. Here, we tested ECTraj with multiple sampling steps, which can sometimes improve metrics in the image domain [5, 6]. However, in our case, the best results are achieved with a single sampling step. In our lower dimensional setting, compared to the image domain, successive noise addition may overly corrupt the samples and in turn compromise their quality, which may be even more evident in low-dimensional data. We report this experiment in Table 4.

1.4 QCNet marginals as priors

ECTraj, similarly to OptTrajDiff [7], uses QCNet marginals as priors to facilitate the conditioning process. Here, we ablate the QCNet marginals to observe how well the model performs without them. The QCNet encoder is still used to

Table 5: Ablation on QCNet marginals as priors

Method	ADE_6	FDE_6	ADE_1	FDE_1	$b-FDE_6$	MR_6	CR_6
OptTrajDiff	0.68	1.36	1.37	3.60	1.99	0.20	0.02
OptTrajDiff + priors	0.60	1.31	1.08	2.71	1.95	0.17	0.01
ECTraj	0.64	1.34	1.52	4.14	2.03	0.19	0.02
ECTraj + priors	0.50	0.96	0.94	2.37	1.68	0.13	0.01

retrieve the scene encoding. Table 5 shows the ablation study on using marginal QCNet predictions as priors. For both OptTrajDiff and ECTraj, marginals are useful to refine predictions. This is especially noticeable with ECTraj, where the improvement in metrics is relatively higher when marginals are applied. One possible reason for this is the single-step sampling nature of ECTraj - more precisely, priors are more informative when we sample for fewer steps (such as one) compared to more steps (such as 10, like in OptTrajDiff). Figure 1 shows some scenarios where introducing the QCNet marginals into conditioning significantly improves predictions.

2 Consistency training details

In this section, we will discuss some technical details of consistency training.

2.1 Linear mapping

For faster training convergence, 120-dimensional (flattened) trajectories are compressed into 10-dimensional latent vectors in line with [2] and [7]. This is done using linear mapping, where the trajectory vectors $\mathbf{x}$ are mapped into latents $\mathbf{z}$ as:

$$\mathbf{z} = \mathcal{G}(x) = \mathbf{x}U. \quad (2)$$

The latents are mapped back to the trajectory space as:

$$\mathbf{x} = \mathcal{F}(z) = \mathbf{z}V. \quad (3)$$

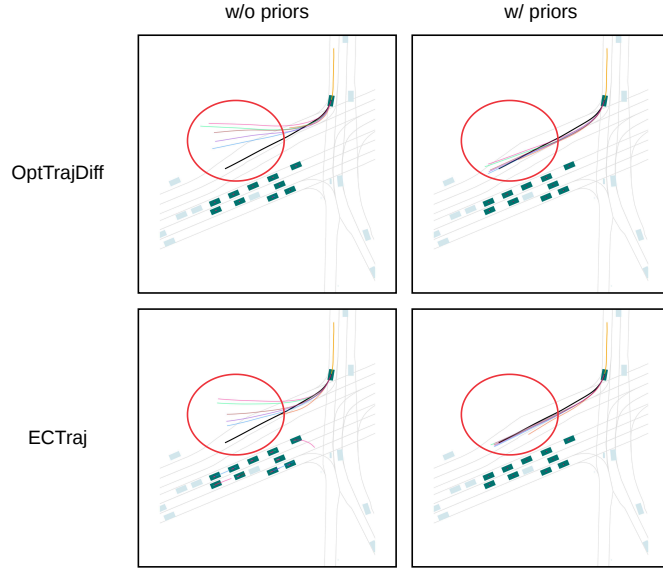
Both U and V are learnable matrices, whose values are obtained by optimizing a composite loss function:

1. **Reconstruction loss** - which minimizes the distance between the ground truth trajectory and its reconstruction:

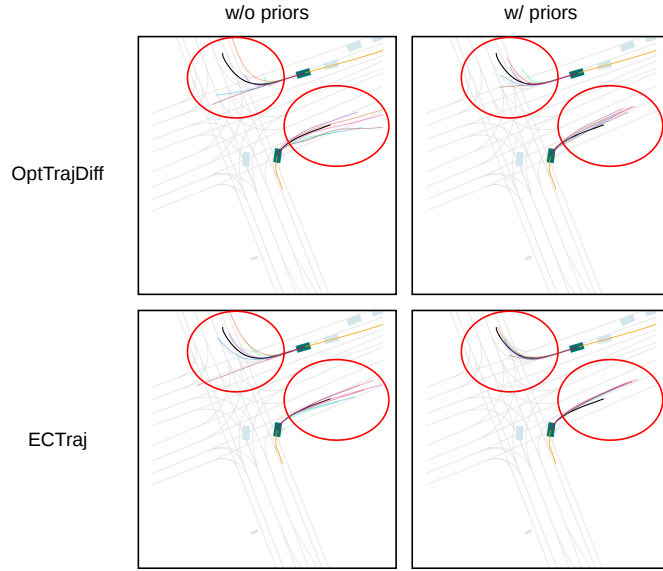
$$\mathcal{L}_{rec} = \|\mathcal{F}(\mathcal{G}(X)) - x\|_2^2 \quad (4)$$

2. **Regularization** - which aims to make the mappings distance-preserving; in other words, smaller latent-space distances should translate to smaller trajectory-space distances and vice versa. This is done as follows:

$$\mathcal{L}_{reg} = (\mathbf{x}^T \mathbf{x} - \mathcal{G}(\mathbf{x})^T \mathcal{G}(\mathbf{x}))^2 \quad (5)$$

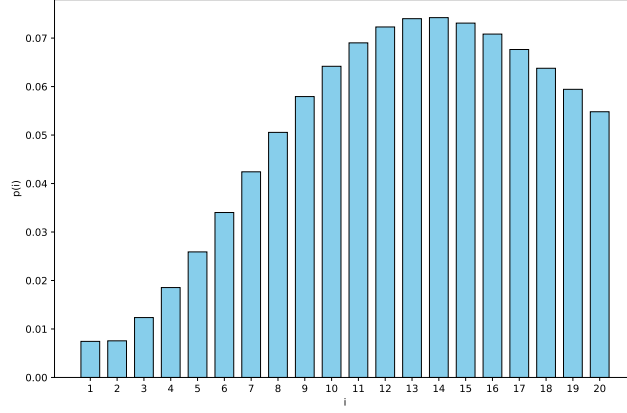


(a) Scenario 1



(b) Scenario 2

Fig. 1: Scenarios which benefit from incorporating the QCNet marginals prior. Regions of interest are marked in **red**. In both scenarios, not incorporating the prior leads to some modes violating environmental constraints (i.e., driving in the non-driveable area) and, also, decreased accuracy. With the prior introduced, ECTraj gives a more accurate prediction compared to OptTrajDiff, particularly in Scenario 1 (a).

Fig. 2: Discrete lognormal time step distribution for $N = 20$

3. **Variance minimization** - which aims to reduce training instability due to extreme variances in some dimensions:

$$\mathcal{L}_{var} = ||std(\mathcal{G}(\mathbf{x})) - \mathbf{v}||_2^2, \quad (6)$$

where $\mathbf{v}$ is a vector whose elements are all η , which is a learnable parameter.

2.2 Lognormal time step distribution

During training, the student index t is sampled from the discrete lognormal time step distribution in the main paper. This distribution is formed in line with [5], and we will provide some additional details here.

First, we recall that each time step index i is associated with a noise variance σ_i . Taking this into consideration, the discrete lognormal distribution the variance is sampled from can be formulated as:

$$\ln(\sigma_i) \sim \mathcal{N}(\mu, \Sigma, N), \quad (7)$$

where μ and Σ are the mean and variance of the distribution. In our case $\mu = -1.1$ and $\Sigma = 2.0$. N denotes the number of PF ODE discretization steps, which is described in the main paper. Based on Equation (7), the distribution of variances σ_i (and with that, time steps i) is defined as:

$$p(i) = p(\sigma_i) = \text{erf} \frac{\ln(\sigma_i) - \mu}{\sqrt{2}\Sigma} - \text{erf} \frac{\ln(\sigma_{i-1}) - \mu}{\sqrt{2}\Sigma}, \quad (8)$$

where erf denotes the Gauss error function and $i \in 0, 1, \dots, N$. We note that obtaining $p(\sigma_1)$ would require us to additionally define σ_0 , which is simply assigned

the value 0 (that is, $\sigma_0 = 0$). $\sigma_1 = \sigma_{min} = 0.002$ and $\sigma_N = \sigma_{max} = 1.0$. All the other values $t \in [2, N - 1]$ are obtained in line with [3]:

$$\sigma_t = (\sigma_{min}^{\frac{1}{\rho}} + \frac{t-1}{N-1}(\sigma_{max}^{\frac{1}{\rho}} - \sigma_{min}^{\frac{1}{\rho}}))^{\rho}, \quad (9)$$

Since we discretize the PF ODE trajectory during training, we need to sample time steps as integers. We visualize the discrete lognormal distribution for $N = 20$ (second stage of training: epochs 21-40) in Figure 2.

2.3 Consistency parameterization

As is mentioned in the main paper, the denoiser f_{θ} is parameterized as follows:

$$f_{\theta}(\mathbf{x}, \sigma, \mathbf{C}) = c_{skip}(\sigma)\mathbf{x} + c_{out}(\sigma)F_{\theta}(\mathbf{x}, \sigma, \mathbf{C}), \quad (10)$$

where $\mathbf{x}$ denotes the input, σ denotes the noise level (variance), and $\mathbf{C}$ is the condition which consists of the scene context c , marginal QCNet predictions mm , and their scores p . F_{θ} denotes the learnable part of our denoiser. This parameterization needs to satisfy the boundary condition:

$$f_{\theta}(\mathbf{x}, \sigma_{min}, \mathbf{C}) = \mathbf{x}. \quad (11)$$

To accomplish this, in line with [6], $c_{skip}(\cdot)$ and $c_{out}(\cdot)$ are designed as differentiable functions for which $c_{skip}(\sigma_{min}) = 1$ and $c_{out}(\sigma_{min}) = 0$. We opted for the following choices, similarly to [6]:

$$c_{skip} = \frac{0.25}{(\sigma - \sigma_{min})^2 + 0.25}, \quad (12)$$

$$c_{out} = \frac{0.5(\sigma - \sigma_{min})}{\sqrt{0.25 + \sigma^2}}, \quad (13)$$

where $\sigma_{min} = 0.002$.

References

1. Geng, Z., Pokle, A., Luo, W., Lin, J., Kolter, J.Z.: Consistency models made easy. In: The Thirteenth International Conference on Learning Representations (ICLR) (2025)
2. Jiang, C., Cornman, A., Park, C., Sapp, B., Zhou, Y., Anguelov, D., et al.: Motion-Diffuser: Controllable multi-agent motion prediction using diffusion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 9644–9653 (2023)
3. Karras, T., Aittala, M., Aila, T., Laine, S.: Elucidating the design space of diffusion-based generative models. *Advances in Neural Information Processing Systems (NeurIPS)* **35**, 26565–26577 (2022)
4. Shen, L., Kwok, J.: Non-autoregressive conditional diffusion models for time series prediction. In: International Conference on Machine Learning (ICML). pp. 31016–31029. PMLR (2023)
5. Song, Y., Dhariwal, P.: Improved techniques for training consistency models. In: The Twelfth International Conference on Learning Representations (ICLR) (2024)
6. Song, Y., Dhariwal, P., Chen, M., Sutskever, I.: Consistency models. In: Proceedings of the 40th International Conference on Machine Learning (ICML). pp. 32211–32252 (2023)
7. Wang, Y., Tang, C., Sun, L., Rossi, S., Xie, Y., Peng, C., Hannagan, T., Sabatini, S., Poerio, N., Tomizuka, M., et al.: Optimizing diffusion models for joint trajectory prediction and controllable generation. In: European Conference on Computer Vision (ECCV). pp. 324–341. Springer (2024)